



Started on	Thursday, 27 August 2026, 11:20 AM
State	Finished
Completed on	Thursday, 27 August 2026, 11:25 AM
Time taken	5 mins 5 secs
Marks	1.00/1.00
Grade	10.00 out of 10.00 (100%)

Question 1 | Correct | Mark 1.00 out of 1.00**Problem Statement**

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

Answer: (penalty regime: 0 %)

```

1  #include<stdio.h>
2  int main(){
3      int m;
4      scanf("%d",&m);
5      int a[m];
6      for(int i=0;i<m;i++){
7          scanf("%d",&a[i]);
8      }
9      int c=0;
10     for(int i=0;i<m;i++){
11         if(a[i]==0){
12             c++;
13         }
14     }
15     printf("%d",c);
16 }
```

	Input	Expected	Got	
✓	5 1 1 1 0 0	2	2	✓
✓	10 1 1 1 1 1 1 1 1 1 1 1	0	0	✓

	Input	Expected	Got	
✓	8 0 0 0 0 0 0 0 0 0	8	8	✓
✓	17 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0	2	2	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)



Joshnavi K B 251801122-2025-AIDS ▾

J2

Started on	Tuesday, 18 August 2026, 11:04 PM
State	Finished
Completed on	Tuesday, 18 August 2026, 11:05 PM
Time taken	57 secs
Marks	1.00/1.00
Grade	10.00 out of 10.00 (100%)

Question 1 | Correct Mark 1.00 out of 1.00

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than $\lfloor n / 2 \rfloor$ times. You may assume that the majority element always exists in the array.

Example 1:

Input: `nums = [3,2,3]`

Output: 3

Example 2:

Input: `nums = [2,2,1,1,1,2,2]`

Output: 2

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 104`
- `-231 <= nums[i] <= 231 - 1`

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer: (penalty regime: 0 %)

```

1  #include <stdio.h>
2
3  int majority(int a[], int low, int high)
4  {
5      if (low == high)
6          return a[low];
7
8      int mid = (low + high) / 2;
9
10     int left = majority(a, low, mid);
11     int right = majority(a, mid + 1, high);
12
13     if (left == right)
14         return left;
15
16     int countLeft = 0, countRight = 0;
17
18     for (int i = low; i <= high; i++)
19     {
20         if (a[i] == left)
21             countLeft++;
22         if (a[i] == right)
23             countRight++;
24     }
25
26     if (countLeft > countRight)
27         return left;
28     else
29         return right;
30 }
31
32 int main()
33 {
34     int n;
```

```
35     scanf("%d", &n);
36
37     int a[n];
38
39     for (int i = 0; i < n; i++)
40         scanf("%d", &a[i]);
41
42     printf("%d", majority(a, 0, n - 1));
43
44     return 0;
45 }
```

	Input	Expected	Got	
✓	3	3	3	✓
	3 2 3			

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)





Joshnavi K B 251801122-2025-AIDS ▾

J2

Started on	Tuesday, 18 August 2026, 11:05 PM
State	Finished
Completed on	Tuesday, 18 August 2026, 11:06 PM
Time taken	52 secs
Marks	1.00/1.00
Grade	10.00 out of 10.00 (100%)

Question 1 | Correct | Mark 1.00 out of 1.00**Problem Statement:**

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer: (penalty regime: 0 %)

```
1  #include <stdio.h>
2
3  int floorValue(int a[], int low, int high, int x)
4  {
5      if (low > high)
6          return -1;
7
8      if (a[high] <= x)
9          return a[high];
10
11     int mid = (low + high) / 2;
12
13     if (a[mid] == x)
14         return a[mid];
15
16     if (a[mid] > x)
17         return floorValue(a, low, mid - 1, x);
18
19     int result = floorValue(a, mid + 1, high, x);
20
21     if (result == -1)
22         return a[mid];
23
24     return result;
25 }
26
27 int main()
28 {
29     int n, x;
30     scanf("%d", &n);
31
32     int a[n];
33
34     for (int i = 0; i < n; i++)
35         scanf("%d", &a[i]);
36
37     scanf("%d", &x);
38
39     printf("%d", floorValue(a, 0, n - 1, x));
40
41     return 0;
42 }
```



	Input	Expected	Got	
✓	6 1 2 8 10 12 19 5	2	2	✓
✓	5 10 22 85 108 129 100	85	85	✓
✓	7 3 5 7 9 11 13 15 10	9	9	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)



Joshnavi K B 251801122-2025-AIDS ▾

J2

Started on Tuesday, 18 August 2026, 11:06 PM**State** Finished**Completed on** Tuesday, 18 August 2026, 11:07 PM**Time taken** 53 secs**Marks** 1.00/1.00**Grade** 10.00 out of 10.00 (100%)

Question 1 | Correct | Mark 1.00 out of 1.00**Problem Statement:**

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

Answer: (penalty regime: 0 %)

```
1  #include <stdio.h>
2
3  int findPair(int a[], int left, int right, int x, int *p, int *q)
4  {
5      if (left >= right)
6          return 0;
7
8      int sum = a[left] + a[right];
9
10     if (sum == x)
11     {
12         *p = a[left];
13         *q = a[right];
14         return 1;
15     }
16
17     if (sum < x)
18         return findPair(a, left + 1, right, x, p, q);
19
20     return findPair(a, left, right - 1, x, p, q);
21 }
22
23 int main()
24 {
25     int n, x, p, q;
26     scanf("%d", &n);
27
28     int a[n];
29
30     for (int i = 0; i < n; i++)
31         scanf("%d", &a[i]);
32
33     scanf("%d", &x);
34
35     if (findPair(a, 0, n - 1, x, &p, &q))
36     {
37         printf("%d\n", p);
38         printf("%d", q);
39     }
40     else
41     {
42         printf("No");
43     }
44
45     return 0;
46 }
```

	Input	Expected	Got	
✓	4	4	4	✓
	2	10	10	
	4			
	8			
	10			
	14			
✓	5	No	No	✓
	2			
	4			
	6			
	8			
	10			
	100			

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Back to Course



Joshnavi K B 251801122-2025-AIDS ▾

J2

Started on Tuesday, 18 August 2026, 11:08 PM**State** Finished**Completed on** Tuesday, 18 August 2026, 11:09 PM**Time taken** 59 secs**Marks** 1.00/1.00**Grade** 10.00 out of 10.00 (100%)

Question 1 | Correct | Mark 1.00 out of 1.00

Write a Program to Implement the Quick Sort Algorithm

Input Format:

The first line contains the no of elements in the list-n

The next n lines contain the elements.

Output:

Sorted list of elements

For example:

Input	Result
5	12 34 67 78 98
67 34 12 98 78	

Answer:

```

1  #include <stdio.h>
2
3  void quickSort(int a[], int low, int high)
4  {
5      if (low < high)
6      {
7          int i = low;
8          int j = high;
9          int pivot = a[low];
10
11         while (i < j)
12         {
13             while (a[i] <= pivot && i < high)
14                 i++;
15
16             while (a[j] > pivot)
17                 j--;
18
19             if (i < j)
20             {
21                 int temp = a[i];
22                 a[i] = a[j];
23                 a[j] = temp;
24             }
25         }
26
27         a[low] = a[j];
28         a[j] = pivot;
29
30         quickSort(a, low, j - 1);
31         quickSort(a, j + 1, high);
32     }
33 }
34
35 int main()
36 {
37     int n;
38     scanf("%d", &n);
39
40     int a[n];
41
42     for (int i = 0; i < n; i++)
43         scanf("%d", &a[i]);
44
45     quickSort(a, 0, n - 1);
46
47     for (int i = 0; i < n; i++)
48         printf("%d ", a[i]);
49
50     return 0;
51 }
```

	Input	Expected	Got	
✓	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✓
✓	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✓
✓	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

[Back to Course](#)